

STRIPS Planning: Tutorials Extra Slides

Possible sticking points, checks, and live-Python debugging

COMP9414 Artificial Intelligence

What To Slow Down On

Don't rush

- ▶ Preconditions are checked before effects.
- ▶ Delete effects really remove facts.
- ▶ Facts not mentioned by effects persist.
- ▶ A plan is invalid at the first illegal action.

Things to consider

- ▶ Ask: "What facts are true right now?"
- ▶ Check one action at a time.
- ▶ Do not let later actions rescue an illegal earlier step.
- ▶ Keep the picture secondary to the facts.

Carefully use update definitions: Unlocking Is not moving

Current facts include:

```
at(robot,corridor),  has(robot,keycard),  locked(lab_door)
```

Apply:

```
unlock_door(robot,keycard,lab_door,corridor,lab)
```

Correct update

Delete `locked(lab_door)`.
Add `unlocked(lab_door)`.
Keep `at(robot,corridor)`.

Incorrect update

Do not add `at(robot,lab)`.
Unlocking changes the door state; it does not move the robot.

Don't forget delete effects

For:

```
pickup_parcel(robot,parcel,mail_room)
```

Important delete effects

- ▶ `at(parcel,mail_room)`
- ▶ `handempty(robot)`

The parcel is no longer in the room, and the robot's parcel hand is no longer empty.

If `handempty(robot)` is not deleted, the model may say both facts are true: `carrying(robot,parcel)` and `handempty(robot)`.

Debugging a broken plan

Use this checklist when you don't know why your plan is rejected.

1. Start from the initial fact set.
2. For the next action, check every precondition.
3. If any precondition is false, stop: this is the first error.
4. Otherwise, remove delete effects.
5. Add add effects.
6. Continue with the next action.

A useful question is: "Which precondition is false in the state immediately before this action?"

Reading PDDL without panicking

For this week, PDDL is just a text format for the same STRIPS ideas.

What to read

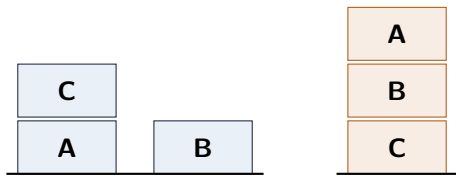
- ▶ `:parameters:` variables.
- ▶ `:precondition:` facts required.
- ▶ `:effect:` facts changed.
- ▶ `(not ...)` inside effects: delete.

What to ignore for now

- ▶ Quantifiers.
- ▶ Conditional effects.
- ▶ Numeric costs.
- ▶ Time and concurrency.
- ▶ Complicated type hierarchies.

Sussman Anomaly: The trap

Initial state:



Goal: $\text{on}(A,B)$ and $\text{on}(B,C)$.

Tempting plan: make $\text{on}(A,B)$ first.

```
unstack(C,A), putdown(C), pickup(A), stack(A,B)
```

Now B is not clear, so $\text{on}(B,C)$ is blocked.

Sussman Anomaly: and the fix

A valid plan interleaves the goals:

1. `unstack(C,A)`
2. `putdown(C)`
3. `pickup(B)`
4. `stack(B,C)`
5. `pickup(A)`
6. `stack(A,B)`

Explanation

- ▶ Achieving one goal can delete or block another.
- ▶ `clear(B)` is needed before `stack(B,C)`.
- ▶ `stack(A,B)` destroys `clear(B)`.
- ▶ So do `on(B,C)` before `on(A,B)`.

Live Python troubleshooting

If the browser rejects the returned plan, check the Python result shape first.

Required fields

- ▶ `algorithm`: exactly `strips_bfs`.
- ▶ `status`: usually `found`.
- ▶ `plan`: list of action strings.
- ▶ `stats`: optional dictionary of counts.

Possible plan errors

- ▶ Action string not grounded.
- ▶ Spelling differs from app action names.
- ▶ Action is legal later, but not when attempted.
- ▶ Solver mutates facts in place and corrupts a queued state.